Operadores físicos: σ, π, δ, γ, $\cup$

Clase 18

IIC2413 - Sección 3

Prof. Cristian Riveros

Outline

Sobre operadores físicos

Selección

Proyección

Duplicados

Group by

Unión

Outline

Sobre operadores físicos

Selección

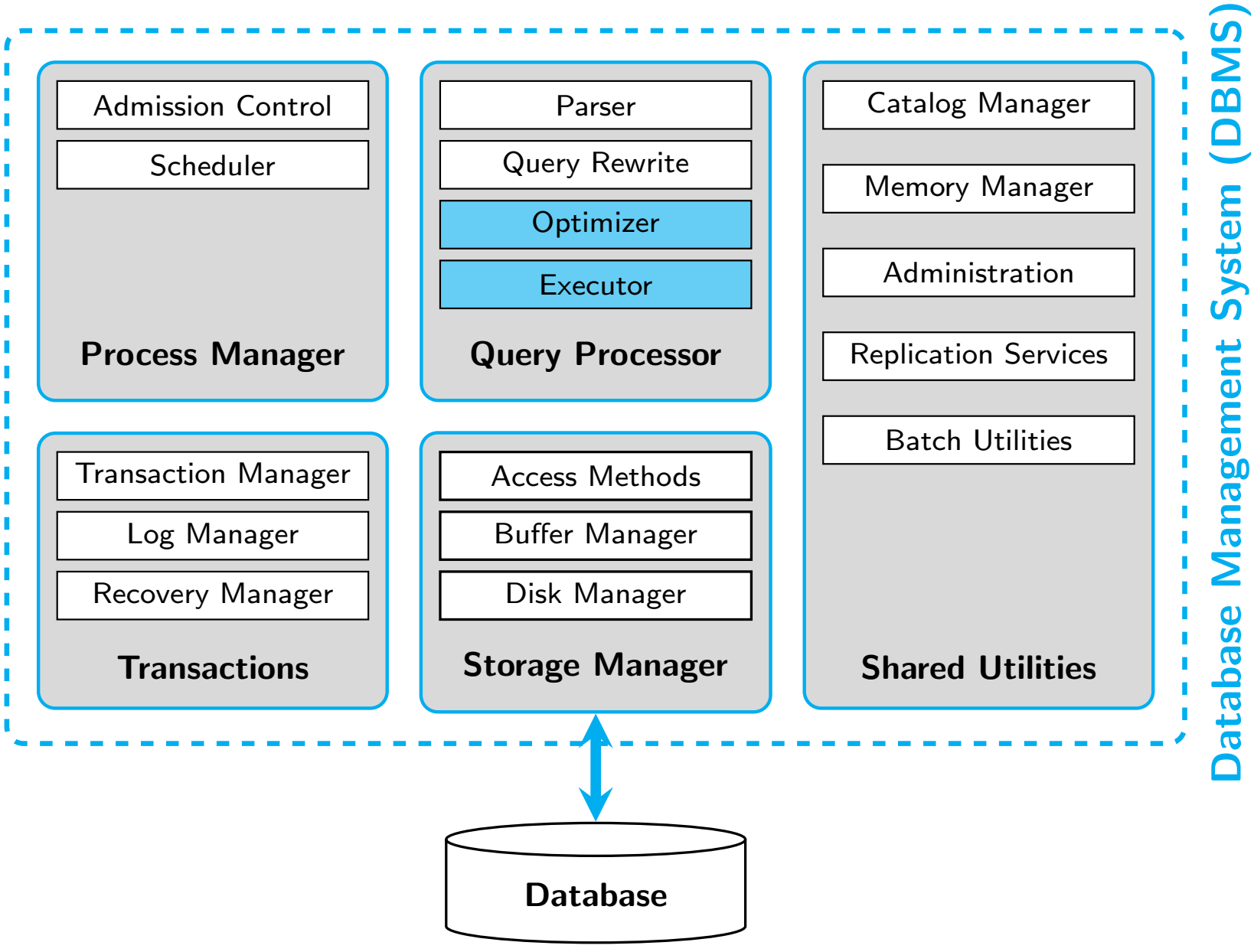
Proyección

Duplicados

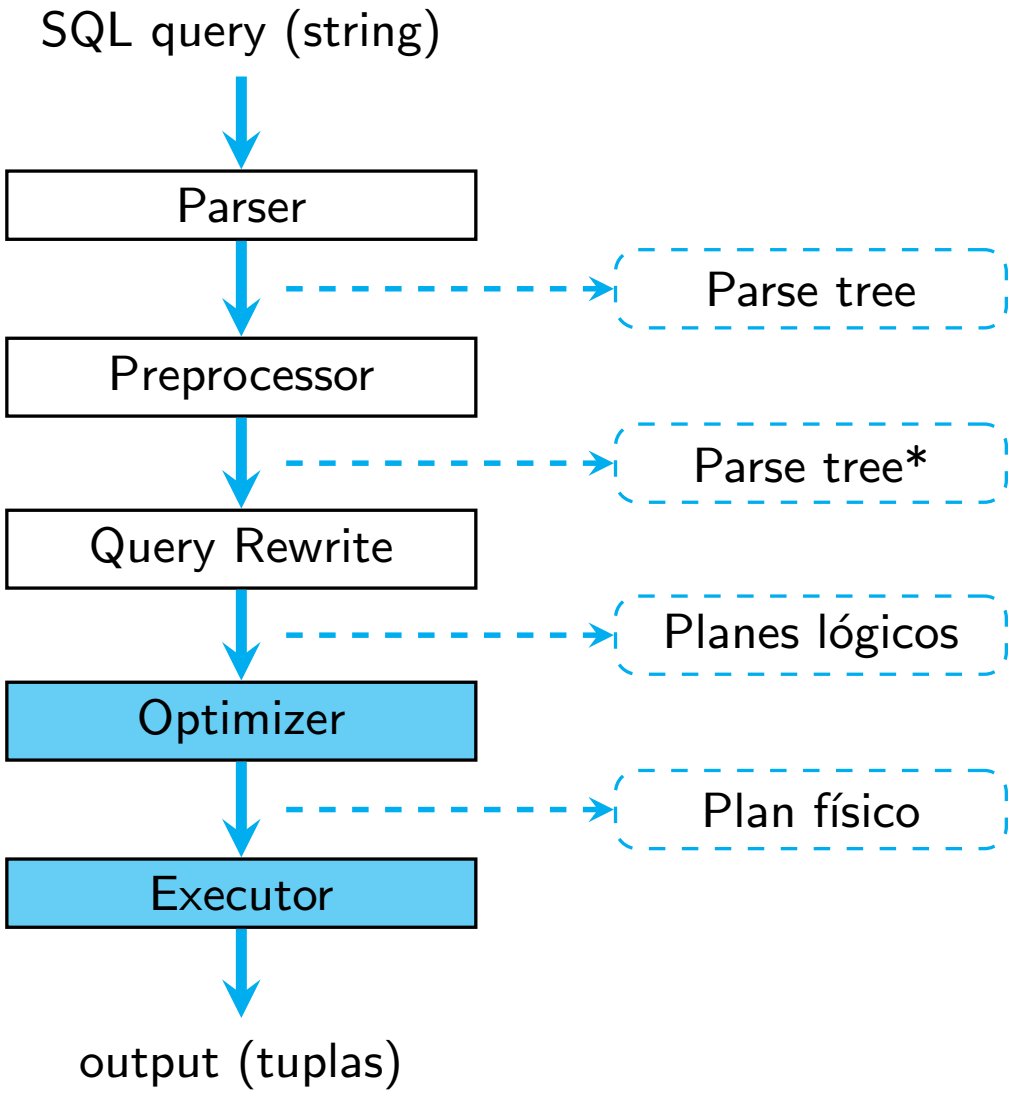
Group by

Unión

Implementación de operadores relacionales



Implementación de operadores relacionales

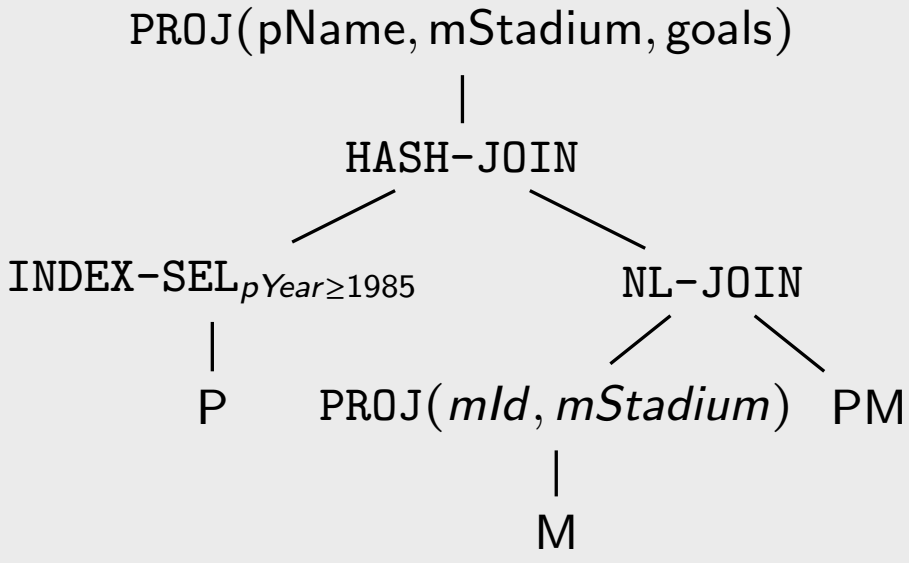


¿qué es un plan físico?

Definición

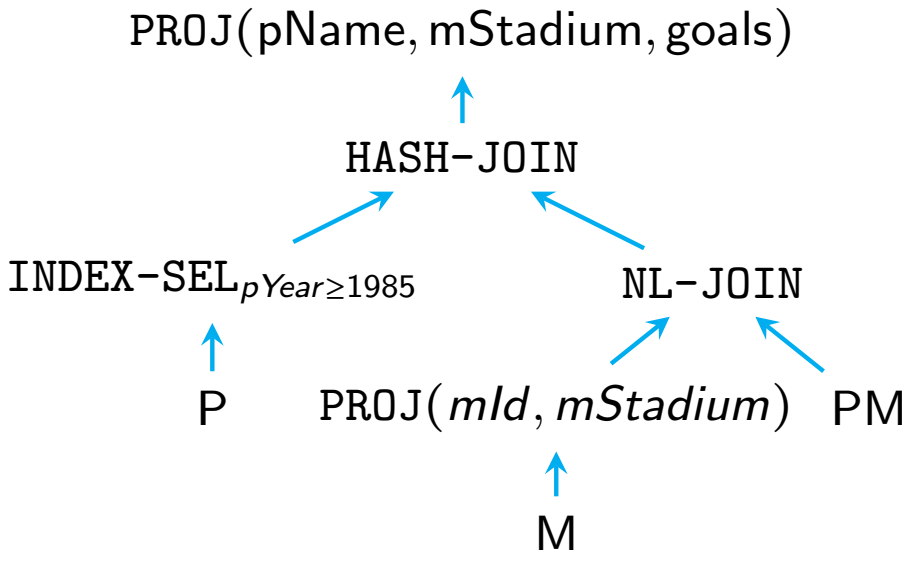
Un **plan físico** es un árbol ordenado y etiquetado, similar a un plan lógico, tal que cada nodo es una **implementación** de un operador relacional.

Ejemplo



¿cómo se ejecuta un plan físico?

Pipelining



- Ejecución **en serie** del plan físico.
- Cada operador genera **una tupla a la vez**.

Cada operador retorna **uno a uno** sus resultados al operador padre

¿cómo se ejecuta un plan físico?

Interfaz de operadores físicos

`open()`

- Inicializa el estado del operador.
- Ajusta parámetros (ej. condición de selección).

`next()`

- Procesa el input y responde la siguiente tupla.

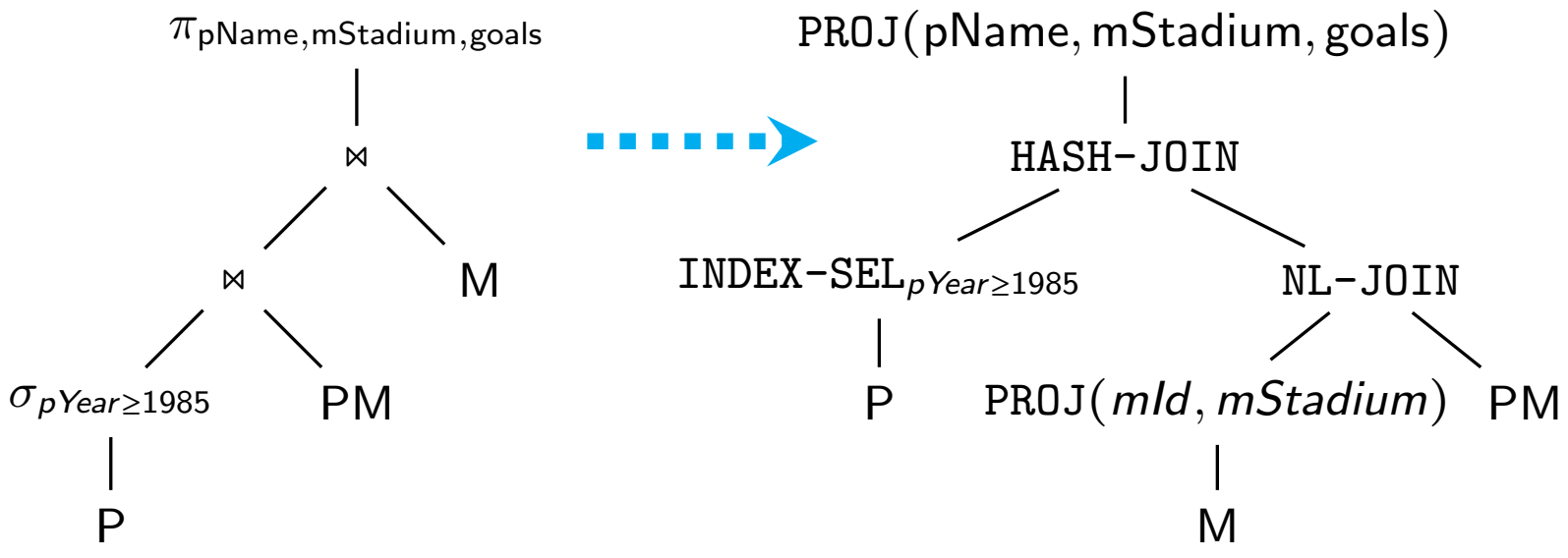
`close()`

- Limpia elementos temporales.

Importante para **componer y ejecutar** el plan físico

¿cómo escogemos el plan físico?

1. Convertir los planes lógicos en planes físicos.
2. Calcular el **costo** de cada plan físico.
3. Escoger el plan con menor costo.



¿cómo calculamos ese costo?

(usaremos I/O a disco)

Parámetros para medir costo

$\text{cost}(R)$:	costo (en I/O) para computar R.
$\text{pages}(R)$:	cantidad de páginas necesarias para almacenar R.
$\lvert R \rvert$:	cantidad de tuplas en R.
$\text{tsize}(R)$:	tamaño promedio de una tupla en R.
$\text{distinct}(R)$:	cantidad de elementos distintos en R.
$\text{distinct}_A(R)$:	cantidad de elementos distintos en el atributo A.
$\text{sel}_\theta(R)$:	$\lvert\sigma_\theta(R)\rvert \,/\, \lvert R \rvert$ (**selectividad** de θ)

Predicados θ

$$\theta \quad := \quad A \sim a \;\mid\; A \sim B \;\mid\; \theta \wedge \theta \;\mid\; \theta \vee \theta$$

- A y B son atributos cualquiera
- $\sim \in \{=, \neq, <, >, \leq, \geq, \ldots\}$

Parámetros para medir costo

$\text{cost}(R)$:	costo (en I/O) para computar R.	
$\text{pages}(R)$:	cantidad de páginas necesarias para almacenar R.	
$\lvert R \rvert$:	cantidad de tuplas en R.	
$\text{tsize}(R)$:	tamaño promedio de una tupla en R.	
$\text{distinct}(R)$:	cantidad de elementos distintos en R.	
$\text{distinct}_A(R)$:	cantidad de elementos distintos en el atributo A.	
$\text{sel}_\theta(R)$:	$\lvert\sigma_\theta(R)\rvert \,/\, \lvert R \rvert$	(**selectividad** de θ)

Estos parámetros debemos **estimarlos** para cada operador

Durante esta clase,
calcularemos los siguientes parámetros

$\text{cost}(R)$:	costo (en I/O) para computar R.
$\text{pages}(R)$:	número de páginas necesarias para almacenar R.
$\lvert R\rvert$:	número de tuplas en R.
$\text{tsize}(R)$:	tamaño promedio de una tupla en R.

Ejemplo

Los parámetros de **sorting** de R (o $\tau(R)$) son:

$$\begin{aligned}
\text{cost}(\tau(R)) &= \text{cost}(R) + 2 \cdot \text{pages}(R) \\
\text{pages}(\tau(R)) &= \text{pages}(R) \\
|\tau(R)| &= |R| \\
\text{tsize}(\tau(R)) &= \text{tsize}(R)
\end{aligned}$$

Durante esta clase,
calcularemos los siguientes parámetros

cost(R):	costo (en I/O) para computar R.
pages(R):	número de páginas necesarias para almacenar R.
$\|R\|$:	número de tuplas en R.
tsize(R):	tamaño promedio de una tupla en R.

Los siguientes parámetros:

distinct(R):	cantidad de elementos distintos en R.	
distinct$_A$(R):	cantidad de elementos distintos en el atributo A.	
sel$_\theta$(R):	$\|\sigma_\theta(R)\| / \|R\|$	(**selectividad** de θ)

los **estimaremos** en otra clase (por ahora los supondremos dados).

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Selección σ_θ

Predicados θ

$$\theta \quad := \quad A \sim a \mid A \sim B \mid \theta \wedge \theta \mid \theta \vee \theta$$

- A y B son atributos cualquiera
- $\sim \in \{=, \neq, <, >, \leq, \geq, \ldots\}$

Varias posibilidades:

1. Sin índices.
2. Con índice primario.
3. Con índices secundarios.
4. Distintos filtros/predicados.

Selección σ_θ: sin índices

1. Leer todas las tuplas, una a una.
2. Retornar la tupla si satisface el predicado.

Algoritmo

input: Predicado θ y relación/operador R.

```
open()
    R.open()

close()
    R.close()
```

```
next()
    t := R.next()
    while t ≠ NULL do
        if t satisface θ then
            return t
        t := R.next()
    return NULL
```

Selección σ_θ: sin índices

Costo sin índices

$$
\begin{aligned}
\mathsf{cost}(\sigma_\theta(R)) &= \mathsf{cost}(R) \\
\mathsf{pages}(\sigma_\theta(R)) &= \mathsf{sel}_\theta(R) \cdot \mathsf{pages}(R) \\
|\sigma_\theta(R)| &= \mathsf{sel}_\theta(R) \cdot |R|
\end{aligned}
$$

Selección σ_θ: índice primario $A = a$

1. Buscamos la primera tupla que satisface $A = a$.
2. Retornar las siguientes tuplas haciendo `next` del índice.

Algoritmo

input: Predicado $A = a$, relación R y índice I.

```
open()
    I.open()
    I.search(A = a)

close()
    I.close()
```

```
next()
    t := I.next()
    if t ≠ NULL then
        return t
    return NULL
```

Selección σ_θ: índice primario $A = a$

Costo con índice primario

$$\begin{aligned}
\text{cost}(\sigma_\theta(R)) &= \underbrace{\text{cost}(I)}_{\approx 3} + \text{sel}_\theta(R) \cdot \text{pages}(R) \\
\text{pages}(\sigma_\theta(R)) &= \text{sel}_\theta(R) \cdot \text{pages}(R) \\
|\sigma_\theta(R)| &= \text{sel}_\theta(R) \cdot |R|
\end{aligned}$$

- $\text{cost}(I)$ depende del tipo de índice.
- Generalmente es igual a 3 pero depende del índice.

Selección σ_θ: índice secundario $A = a$

1. Buscamos la primera tupla que satisface $A = a$.
2. Por cada **data entry** $k*$ que satisface $A = a$:
   - Buscamos la página en $k^*.RID$.
   - Retornamos la tupla con RID igual a $k^*.RID$.

Algoritmo

input: Predicado $A = a$, relación R y índice I.

```
open()
    I.open()
    I.search(A = a)

close()
    I.close()
```

```
next()
    k* := I.next()
    if k* ≠ NULL then
        return R.get(k*.RID)
    return NULL
```

Selección σ_θ: índice secundario $A = a$

Costo con índice secundario

$$\begin{aligned} \text{cost}(\sigma_\theta(R)) &= \underbrace{\text{cost}(I)}_{\approx 3} + \text{sel}_\theta(R) \cdot |R| \\ \text{pages}(\sigma_\theta(R)) &= \text{sel}_\theta(R) \cdot \text{pages}(R) \\ |\sigma_\theta(R)| &= \text{sel}_\theta(R) \cdot |R| \end{aligned}$$

- Notar la diferencia con el caso anterior:

$$\text{sel}_\theta(R) \cdot \text{pages}(R) \ll \text{sel}_\theta(R) \cdot |R|$$

- Si $\text{sel}_\theta(R)$ tiende a 0, entonces la **diferencia de costo** es mínima.

Selección σ_θ: índice primario/secundario

¿qué ocurre si A es una **llave**?

Costo de índice sobre una llave

$$\begin{aligned} \text{cost}(\sigma_\theta(R)) &= \underbrace{\text{cost}(I)}_{\approx 3} \\ \text{pages}(\sigma_\theta(R)) &= 1 \\ |\sigma_\theta(R)| &= 1 \end{aligned}$$

¡Lo importante de una buena **normalización**!

Selección σ_θ: índice primario/secundario $A \leq a$

Mismos costo/parámetros anteriores, pero solamente posible si índice soporta **consultas por rango** *(B+-tree).*

Recordar

- Hash index **no soporta** consultas por rango.
- Costo será el mismo que hacer una selección sin índice.

Selección σ_θ: caso general

1. Predicados **conjuntivos**:

$$\overbrace{\underbrace{(A_1 \sim_1 a_1)}_{\text{termino}} \texttt{ AND } \cdots \texttt{ AND } (A_k \sim_k a_k)}^{\text{conjunción}}$$

2. Predicados **disjuntivos**:

$$\overbrace{\underbrace{(A_1 \sim_1 a_1)}_{\text{termino}} \texttt{ OR } \cdots \texttt{ OR } (A_k \sim_k a_k)}^{\text{disyunción}}$$

3. Predicados booleanos sin restricciones.

Selección σ_θ: predicados conjuntivos

$$p \;:=\; (A_1 \sim_1 a_1) \;\texttt{AND}\; \cdots \;\texttt{AND}\; (A_k \sim_k a_k)$$

1. Hacemos un **scan** sobre toda la relación y filtramos tuplas.
2. Escogemos la conjunción $p_i := (A_i \sim_i a_i)$ que:
   - tenga un **índice** sobre A_i y
   - con **mayor selectividad** (menor $\text{sel}_{p_i}(R)$).

   Las tuplas retornadas por el índice para p_i las filtramos con p.
3. Tratamos de usar **varios índices** a la vez.
   - Si la selectividad de cada predicado es baja.
   - Todos los índices son unclustered/secundarios (¿por qué?).

Selección σ_θ: predicados disyuntivos

$$(A_1 \sim_1 a_1) \texttt{ OR } \cdots \texttt{ OR } (A_k \sim_k a_k)$$

1. si **al menos un termino NO coincide** con ningún índice,
   entonces hacemos scan de toda la relación. (¿por qué?)

2. si **cada termino coincide** con al menos un índice, entonces:
   - Evaluar cada termino por separado.
   - Hacer la unión de todos los resultados.
   - Eliminar duplicados.

Selección σ_θ: predicados booleanos sin restricciones

¡Combinación de las anteriores!

Algunas heuristicas

1. Buscar un termino **disyuntivo** que nos obligue a realizar un scan.
2. Buscar un termino **conjuntivo** que tenga mayor selectividad.

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Proyección π_L

Proyección π_L es muy sencillo (especial para **pipeline**):

1. Leemos las tuplas de R, una a una.
2. **Proyectamos** en los atributos L.

Algoritmo

input: Lista de atributos L y operador R.

```
open()
    R.open()

close()
    R.close()
```

```
next()
    t := R.next()
    if t ≠ NULL then
        return t.project(L)
    return NULL
```

Proyección π_L

Costo proyección

$$
\begin{aligned}
\text{cost}(\pi_L(R)) &= \text{cost}(R) \\
\text{pages}(\pi_L(R)) &= \frac{\text{tsize}(\pi_L(R))}{\text{tsize}(R)} \cdot \text{pages}(R) \\
|\pi_L(R)| &= |R| \\
\text{tsize}(\pi_L(R)) &= \sum_{A \in L} \mathbb{E}(|\pi_A(R)|)
\end{aligned}
$$

Esta proyección NO considera **eliminación de duplicados**

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Eliminación de duplicados δ

Dos posibles implementaciones:

1. Basado en **sorting**
2. Basado en **hashing**

Eliminación de duplicados δ: sorting

1. Hacemos **external merge-sort** de las tuplas.
2. En el último paso, filtramos según la última tupla leída.

Algoritmo

input: Operador R.

```
open()
    R' := merge-sort(R)
    R'.open()
    t := NULL

close()
    R'.close()
```

```
next()
    t' := R.next()
    while t' ≠ NULL do
        if t ≠ t' then
            t := t'
            return t
        t' := R.next()
    return NULL
```

Eliminación de duplicados δ: sorting

Costo de eliminación de duplicados con sorting

$$
\begin{aligned}
\mathsf{cost}(\delta(R)) &= \mathsf{cost}(R) + 2 \cdot \mathsf{pages}(R) \\
\mathsf{pages}(\delta(R)) &= \mathsf{distinct}(R) \cdot \frac{\mathsf{tsize}(R)}{|\mathsf{page}|} \\
|\delta(R)| &= \mathsf{distinct}(R) \\
\mathsf{tsize}(\tau(R)) &= \mathsf{tsize}(R)
\end{aligned}
$$

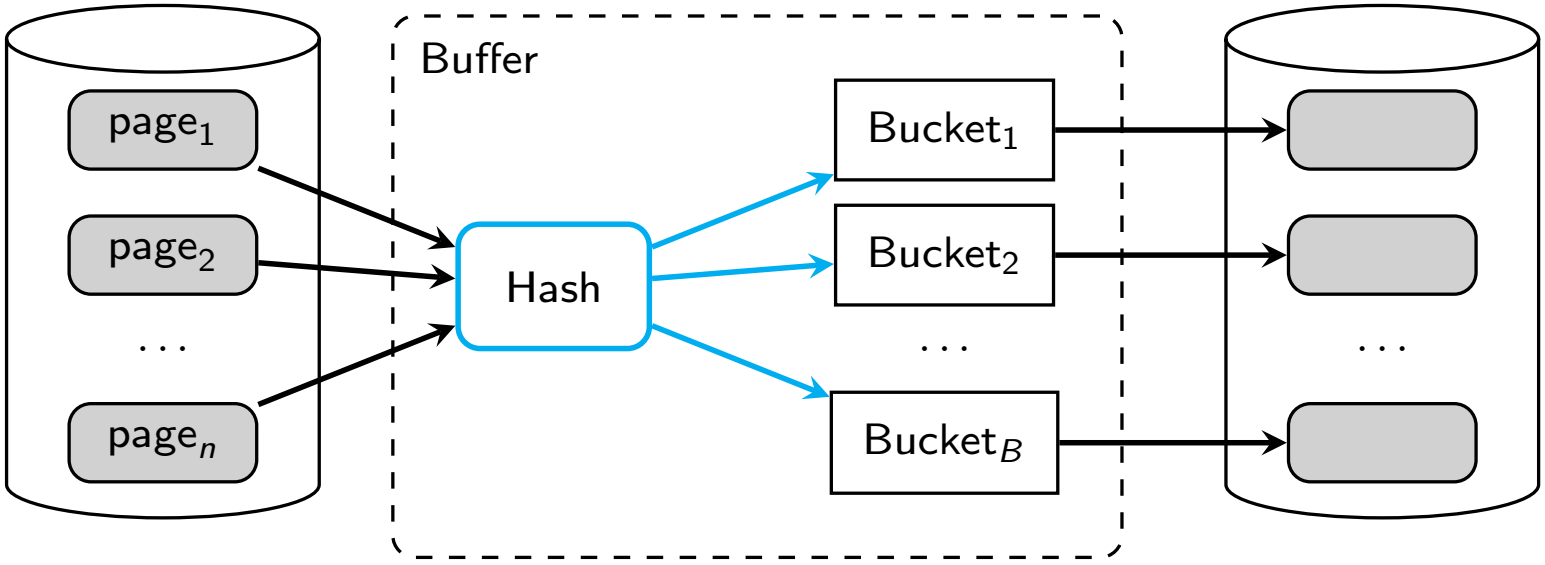
Eliminación de duplicados δ: Hashing

Dos fases

1. Fase de partición
2. Fase de eliminación de duplicados

Eliminación de duplicados δ: Hashing

1. Fase de partición



Eliminación de duplicados δ: Hashing

1. Fase de partición
   - Cada página p del operador R se lee a memoria.
   - Por cada $t \in p$ se computa $h(t)$ y se envía al bucket $\text{Bucket}_{h(t)}$.
   - Si un bucket esta **completo**, se vacía y **materializa** en disco.
   - Cada bucket construye una **lista de páginas** en disco.

Eliminación de duplicados δ: Hashing

1. Fase de partición.
2. Fase de eliminación de duplicados.
   - Cada bucket b se lee a memoria.
   - Se eliminan los duplicados con un algoritmo de memoria interna.
   - Si bucket b no cabe en memoria,
     se itera la fase de **partición** con b.

¿cuánto es la **cantidad optima de buckets** a utilizar?

Eliminación de duplicados δ: Hashing

Costo de eliminación de duplicados con Hashing

$$
\begin{aligned}
\text{cost}(\delta(R)) &= \text{cost}(R) + 2 \cdot \text{pages}(R) \\
\text{pages}(\delta(R)) &= \text{distinct}(R) \cdot \frac{\text{tsize}(R)}{|\text{page}|} \\
|\delta(R)| &= \text{distinct}(R) \\
\text{tsize}(\tau(R)) &= \text{tsize}(R)
\end{aligned}
$$

Similar a la versión basada en **sorting**

Eliminación de duplicados δ: sorting vs. Hashing

¿qué ventajas tiene sorting?

- Los resultados quedan **ordenados**.
- No se ve afectado por datos sesgados.

¿qué ventajas tiene Hashing?

- Posibilidad de usar más buffer.

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

GroupBy γ

Dos posibles implementaciones:

1. Basado en **sorting**.
2. Basado en **hashing**.

Mismos algoritmos **para eliminación de duplicados** pero con agregación

(¿cómo lo hacemos?)

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Operador de unión $\cup$

Dos operadores para unión:

- `UNION`
- `UNION ALL`

¿cuál es la diferencia?

UNION ALL: $R \cup S$ con duplicados

Dos alternativas

1. Si R y S **NO** están ordenados: leemos ambos y retornamos.
2. Si R y S están **ordenados**: leemos, ordenamos y retornamos.
   - Hacemos **merge** de R y S.
   - No aumenta el costo I/O del operador.

UNION ALL: $R \cup S$ con duplicados

Costo de UNION ALL

$$
\begin{aligned}
\text{cost}(R \cup S) &= \text{cost}(R) + \text{cost}(S) \\
\text{pages}(R \cup S) &= \text{pages}(R) + \text{pages}(S) \\
|R \cup S| &= |R| + |S| \\
\text{tsize}(R \cup S) &= \text{tsize}(R)
\end{aligned}
$$

UNION: $R \cup S$ sin duplicados

1. Basado en **sorting**
   - Ordenamos ambas relaciones con sorting externo.
   - Hacemos merge de ambas relaciones.
2. Basado en **Hashing**
   - Particionamos R y S con una función de hash.
   - Eliminamos duplicados en cada partición.

Mismo costo que para **eliminación de duplicados**

Operadores físicos relacionales

Selección (σ)

Proyección (π)

Join ($\bowtie$)

Intersección ($\cap$)

Unión ($\cup$)

Elim. duplicados (δ)

Sorting (τ)

GroupBy (γ)

Próxima clase: join y intersección